ENGG1330: Group Project (2026–27)

Project Release Date: Tuesday, 22 September 2026 | **Submission Deadline:** Monday, 16 November 2026 (23:59)

Build a Puzzle Adventure

Design a **single-player, turn-based console puzzle adventure** in Python. Choose your own setting: for example, explore a mysterious laboratory, restore power to a stranded spaceship or uncover clues in a museum. The player makes choices, discovers information and solves puzzles to reach a clear goal.

Aim for a complete playthrough of about **5–10 minutes**. Include **at least two connected puzzles**: solving one changes what the player can do next. Provide instructions, a clear win condition and a way to restart. A restart may simply mean exiting and running the program again. Numbered choices or typed commands are sufficient. Your game must work offline in a terminal; real-time controls, graphical windows, sound and multiplayer networking are outside this project.

Build a small working version first, then add features. Choose features that serve the game. Simple text and numbered choices are sufficient; keep input within the terminal area specified below.

Your Group and Dates

Work in a group of **3–4 within the same tutorial subclass**. Agree on the members and tell your STA during T2. The TA records your group; you do not need to register it yourself. Ask your STA for help finding teammates. Students who miss the tutorial and remain without a group cannot complete the group project. Choose one submission coordinator; all members should check the final submission.

Item	Requirement
Project weight	**16% of the course mark**
Final submission	**16 November 2026**
Selected-project game show	**30 November 2026**

Follow the exact submission time and links announced on Moodle/Ed. Late submissions are not accepted under the course rules; contact the Head TA promptly if serious circumstances affect your work. Plan for approximately 15–30 hours per member, including testing, documentation and videos.

Run in the Course Container

Your game must run in the **provided Docker environment** without additional packages or configuration changes. You may develop elsewhere, but test your submission in a fresh course container. Submit source and assets, not a custom image.

During T2, use the setup guide to run the starter on one group member's laptop. The current environment is `engg1330-project:2026-v03`. Files saved in the shared `project` folder remain on the laptop after the container exits.

Use Python's standard library and the packages supplied in the starter; the T2 handout introduces their uses. Request other packages on Ed; only additions to the shared environment are permitted. The package-request cutoff and final version will be announced on Ed.

Your game must keep all drawing, messages and input within an **80-column by 24-row terminal area**. Do not write outside this area or allow input to wrap or scroll the display beyond it. Larger terminals may be used, but the game must not require extra space. If the terminal is smaller than 80 × 24, pause play and show a resize message that fits the available space.

What to Submit

One member submits the agreed final version on Moodle for the whole group.

Deliverable	Contents
Code ZIP	All Python source and local assets required to run the game, plus `README`, acknowledgements and `ai_use.md`.
README	Game purpose and rules; names, and subclass; container version; exact run command; tests and known limitations; contributions and sources. Include it in the ZIP.
Trailer, under 1 minute	A publicly viewable video link showing the game's purpose and highlights through actual gameplay.
Demo/technical video, under 5 minutes	A publicly viewable link showing an edited playthrough of the main puzzle sequence and explaining important implementation choices. Explain the logic rather than reading code aloud. **Include a brief development reflection:** groups using AI should explain how they checked, verified, or improved an AI suggestion; groups not using AI should discuss key development lesson learned during the project.

Keep the game at the top level of the extracted `project` folder. Prefer `main.py` as the entry point; otherwise state the exact command. Include necessary assets using relative paths. The grader must not need your accounts, additional downloads or access to files elsewhere on your laptop. Both videos must show the submitted version. Explain the game clearly; elaborate editing is unnecessary.

Before submitting, extract the ZIP into a separate copy of the starter's `project` folder, run it in a fresh course container and follow your own instructions. Check normal play, a wrong action, a boundary case, the win condition and restarting. Confirm the video links open without requesting access. All members should agree that this is the version to submit. Later working-folder edits do not change the submitted ZIP.

Your submission will be shared with classmates during the Peer Review period, so exclude private data and credentials.

Collaboration, Sources and AI

Collaboration within the group is expected. Outside code and generative AI may be used with **full acknowledgement and disclosure**. Explain what your group created, borrowed or adapted. Cite sources for code, data and other assets and follow their licences. Your group remains responsible for correctness and for understanding the submitted work. Project permissions do not relax individual-assignment or tutorial-challenge restrictions.

Include `ai_use.md`, beginning with **"Generative AI used: Yes"** or **"Generative AI used: No."** If no member used AI, the declaration is sufficient. Otherwise, keep a log of substantive uses as you work:

- Member, date
- Tool and model (where available)
- Purpose
- Substantial parts affected (files/materials)
- What was kept, changed, or rejected, and how results were checked (tests or corrections)

Full prompts or chat transcripts are **not required**. Paid tools are neither required nor rewarded.

Include assistance with design, code, debugging, documentation and videos. Keep the evidence accessible to the teaching team, without credentials or unrelated personal information. The log is a required disclosure record with **no separate mark**. Unacknowledged use is handled under the course's academic-integrity procedure. Disclosed use alone does not earn or lose marks. Your group must understand and explain the work.

`ai_use.md` Template Format:

```
# AI Use
Generative AI used: Yes / No (choose one)

If no group member used AI, no entries are needed. Otherwise add a concise entry for
each substantive use

## Entry
- Member and date:
- Tool and model/version:
- Purpose:
- Substantial parts affected (files/materials):
- What was kept, changed, or rejected, and how results were checked (tests or
corrections):

Keep evidence accessible to the teaching team. Remove credentials and unrelated
personal information. Submit the completed file as ai_use.md with the project.
```

Progress and Contributions

Share planning, implementation, testing and explanation. Contributions can include design, documentation and video production; everyone need not write the same amount of code. Keep a brief contribution record in the README and agree on small tasks and next steps.

During tutorial check-ins, show the current version, explain what has changed, describe each member's contribution and identify a difficulty or next step. Bring unfinished work and questions. The teaching team will announce the schedule; check-in and review activities are completed by the end of their tutorial session.

A group normally receives one project mark. If a review indicates no contribution from a member, staff follow up with the student and group. The student is notified and may respond. Confirmed non-contribution without approved circumstances results in removal from the group and **zero for the project component**. Raise difficulties early.

Peer Reviews and Game Show

Inter-group and intra-group reviews take place in tutorials. For assigned games, play before commenting and give specific feedback about what works, what is unclear and what could improve. For your own group, describe actual contributions with factual examples, including non-coding work. Review allocations and instructions will be announced. Project files will be shared with classmates, so exclude private data and credentials.

Selected projects will be showcased on **30 November 2026**. Game-show voting determines **prizes, not project marks**. Selection is not required for a strong grade.

Assessment

Two TAs assess each project and their marks are averaged. STAs support development and record progress. The percentages below are within the project mark, which contributes 16% to the course.

Criterion	Weight	Evidence
Trailer	15%	Clear purpose and distinctive features, supported by gameplay footage.
Demo and technical explanation	15%	Working demonstration and accurate explanation of important logic and design choices; includes a brief development reflection (AI validation or development lesson).
Technical design and challenge	15%	Meaningful programming, appropriate course concepts and justified design. Complexity serves the game.
Correctness and robustness	15%	Rules work; normal, boundary and invalid actions are handled and tested.
Originality and thoughtful adaptation	15%	Purposeful puzzles and adaptations; the group's work is distinguished from borrowed/generated material.
Readability and documentation	20%	Clear organisation, names and useful comments; reproducible README, tests, contribution record and acknowledgements.
Player experience	5%	Clear instructions, useful feedback and a coherent, playable route to the goal. Assessed by classmates during peer reviews.
Total	**100%**	

Assessors judge the quality of the work against these criteria. Extra features, libraries, ASCII art or animation do not automatically raise the grade.

Detailed Grade Expectations

Grade	Standard	Expectations
A	**Excellent result**	**Game Trailer:** Comprehensive and clear explanation of concept, objectives, and unique features with actual gameplay footage and voice-over. **Demo Video:** Complete playthrough from start to finish with voice-over; in-depth design/programming explanation; includes a clear, insightful development reflection (AI verification/improvement or key development lesson). **Technical Design & Challenge:** Demonstrates advanced Python concepts (e.g. nested data structures, modular function structure) **Correctness & Robustness:** Runs without errors or crashes; comprehensively handles normal, boundary, and invalid player inputs/actions. **Originality & Thoughtful Adaptation:** Introduces unique puzzle mechanics or thoughtful adaptations that clearly distinguish the group's work. **Readability & Documentation:** Exceptionally organized code with clear naming and comments; reproducible README with complete contribution record and acknowledgements.
B	**Good result**	**Game Trailer:** Clear explanation of concept and key features with actual gameplay footage. **Demo Video:** Complete playthrough from start to finish; clear explanation of logic; includes a solid development reflection. **Technical Design & Challenge:** Effective use of course programming concepts (data structures, functions, control flow) to build functional game logic. **Correctness & Robustness:** Runs reliably with minimal minor issues; handles common inputs, boundary conditions, and invalid actions effectively. **Originality & Thoughtful Adaptation:** Purposeful puzzle mechanics and well-integrated adaptations. **Readability & Documentation:** Well-structured code with consistent naming and helpful comments; README and contribution record is completed but lacks some minor details
C	**Satisfactory result**	**Game Trailer:** Basic explanation of concept with actual gameplay footage. **Demo Video:** Complete playthrough with minimal technical explanation; brief development reflection. **Technical Design & Challenge:** Standard use of basic Python structures and functions without deep nesting. **Correctness & Robustness:** Runs generally well but may crash on unhandled edge cases or unusual invalid actions. **Originality & Thoughtful Adaptation:** Standard game mechanics with minor personal adaptations.

Grade	Standard	Expectations
		Readability & Documentation: Moderately readable code; basic comments; adequate README covering essential setup.
D	**Pass / Barely satisfactory**	**Game Trailer:** Unclear or confusing explanation of game concept. **Demo Video:** Incomplete demonstration of gameplay or missing technical reflection. **Technical Design & Challenge:** Minimal technical depth; relies on primitive variables or unorganized code blocks. **Correctness & Robustness:** Frequent errors or crashes when encountering invalid input or boundary conditions. **Originality & Thoughtful Adaptation:** Direct copy or minimal effort without meaningful original contribution. **Readability & Documentation:** Poorly organized code lacking comments; minimal or incomplete README.
F	**Unsatisfactory / Fail**	**Game Trailer:** Missing or fails to show gameplay. **Demo Video:** Missing or does not demonstrate a working game. **Technical Design & Challenge:** Absence of basic programming structures; non-functional logic. **Correctness & Robustness:** Program fails to run or crashes immediately. **Originality & Thoughtful Adaptation:** Unattributed submission or severe plagiarism. **Readability & Documentation:** Unreadable code without comments; unhelpful README.